CS 360
Programming Languages
Day 16

Review

- Racket is a functional language
- Closures, recursion, currying highly encouraged!
- Mutation / side-effects highly discouraged!

Domain-Specific Languages

- aka, “little languages”
- Example: HTML, SQL,
- Includes *markup languages* (formatting), *modeling languages* (describing specifications), *programming languages*, etc
- Perl was meant as a text-processing language but later became “general purpose”

Language-Oriented Programming

- Problem: how do you document code well? See Scribble
- Why LOP? “minimum notation, maximum precision”
- Why Racket? Racket has **macros**. More on those later!
- LOP and DSLs are tools in a toolkit, and have huge potential!

Embedded Languages

- You might think you don’t do or use language-oriented programming
- Ever heard of **printf** or **f-strings**?
- Better yet, ever heard of regular expressions?
-
```
# This regex is a little language embedded in Python
pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
```
- Or even: mathematical expressions!

What makes a language?

- A notation system recognizable/usable by both humans and computers (the **syntax** and **semantics**, the structure and the meaning)

- Sometimes, we use languages to talk to computers (Racket)
- Sometimes, computers use languages to talk to us (stack traces / errors)
- Sometimes, computers talk to each other with languages (JSON)

The Limits of Language

- (aka, Computational Theory in 90 seconds)

- “Turing machines” are theoretical machines capable of solving a huge array of computational problems (boring and interesting!)
- Any machine/language which can solve the same problems as a Turing machine is “Turing complete”
- While a Turing machine can theoretically do everything Python can do, Python is *much* easier to program than a TM
- Not all languages are Turing complete: ex, regular expressions

A Quick Example: L-systems

- You mean there was a language to make L-systems this whole time? Whoops!

- Neat!

Beautiful Racket

- Pay for the book!
- Make sure you've installed beautiful racket!
- Make sure you can run stacker!

Stacker

- A “stack calculator” reads “commands”, one by one, pushing them on to the stack if they are numbers and popping elements and computing their result if they are operations
- See notes/stacker/example_program1.rkt
- This uses “Reverse Polish Notation”, a type of post-fix notation
- As opposed to our standard (natural?) “in-fix notation” or Racket’s characteristic “pre-fix notation”

How does it work?

- At a very high level, we're going to take text and interpret it as Racket code
- This requires *reading* the text (aka **source**) in the correct way (defining a **syntax**)
- And it requires *executing* the read text in the correct way (defining a **semantics**)
- These functions will be called the **reader** and the **expander**

S-Expressions: “Datums” vs Syntax

- `‘my-symbol` is a “datum” or a “symbol” in Racket

- But so is `‘(+ 1 1)`, even though, if it weren’t a datum, it would be a Racket function

- An s-expression is either:
  - An atom, *x*
  - An expression *(x . y)* ***– a pair***

- All lists, even all functions, are s-expressions!

- We can convert datums into actual syntax!

Modules

- Every rkt file you've ever made has been a **module**
- Modules typically start with `#lang module-lang-here`
- The language defines the proper syntax for the module
- See **notes/stacker/cake.rkt** for alternative module-defining syntax
- Our reader/expander will either be one module or two separate modules, depending on context

The Reader

- Remember: the reader should consume some text and read it as Racket code, somehow
- See **notes/stacker/first_reading.rkt**
- See **notes/stacker/second_reading.rkt**
- How will the reader do this?
- See **notes/stacker/first_reader.rkt** and **notes/stacker/example_program2.rkt**
- Note that the reader produces a module!

The Reader and Expander

- See **notes/stacker/stacker.rkt** for the full reader/expander code

The Reader: To-Do

- Define the **src-datums**: what should source lines look like as Racket code?
- Define the **module-datum**: what should the source file look like as a Racket module?
- To Learn:
  - “datum” symbol, ‘: “the following is a datum”
  - “insert” symbol, ~: “replace the following when formatting”
  - “quasiquote” symbol, ` : just like ‘ except a comma denotes actual code
  - “unquote” symbol, , : “the following should be treated as code”
  - “unquote-splicing” symbol, ,@: “splice in a list into this datum”
- See **notes/stacker/new_symbols.rkt**